

NativeMEM: Native Memory Compression for Long-Horizon Robotic Manipulation

Ziye Wang¹ Modi Shi² Chaojun Ni³ Jiazhi Yang⁴ Mengdi Li⁵ Zhizhong Su⁵
Tianwei Lin⁵ Hongyang Li^{1†}

¹ The University of Hong Kong ² Beihang University ³ Peking University
⁴ The Chinese University of Hong Kong ⁵ Horizon Robotics
<https://opendrive-lab.com/NativeMEM>

Abstract: How can pretrained Vision-Language-Action (VLA) models retain long-horizon visual histories with high-frequency updates without sacrificing efficiency? Existing approaches rely on external memory management, which restrains either the memory horizon or the reactivity of pretrained policies. To this end, we present NATIVEMEM, a VLA policy that features long-term and real-time updated memory. At its core is an efficient memory encoding scheme, Native Memory Compression, which repurposes the VLA’s own vision encoder to compress each historical frame from each camera view into a single token. Appended to the input sequence, these memory tokens enable the pretrained VLA to attend over long-term history with negligible latency overhead, requiring neither an external planner nor a freshly initialized memory module. To align the memory tokens with the pretrained policy, we first develop a generic memory tokenizer under the supervision of a frozen VLA on memory-demanding data, and then unfreeze the VLA for task-specific fine-tuning. NATIVEMEM consistently outperforms prior methods, boosting success rates from 32.4% to 84.0% in simulation and up to 98.7% on real robots, while maintaining low inference latency and GPU memory usage. Notably, NATIVEMEM exhibits high data efficiency by achieving competitive results with prior arts using only 20% of the training data.

Keywords: VLA Models, Memory Modeling, Long-Horizon Manipulation

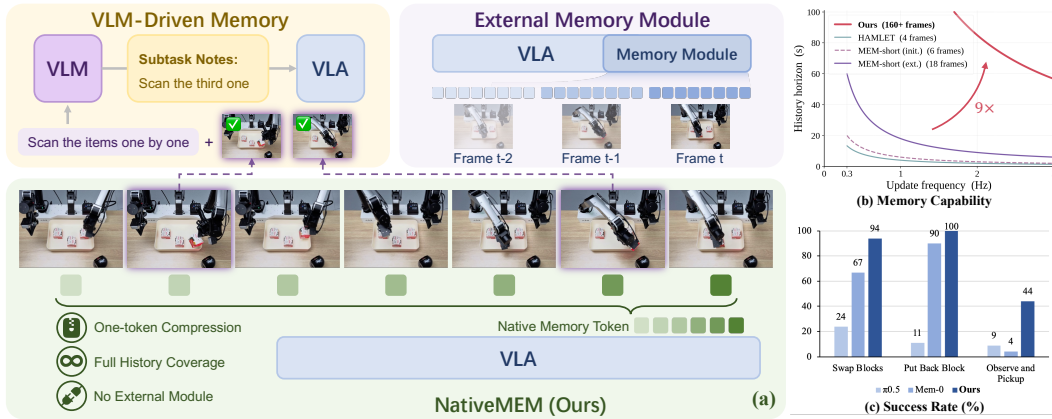


Figure 1: **NATIVEMEM** differs from prior memory-augmented VLAs that rely on VLM-generated textual notes or external memory modules. (a) By repurposing the VLA’s own vision encoder, it compresses each historical frame-view observation into a single native memory token, allowing the policy to condition on the full visual history through its original token sequence. (b) This ultra-compact representation enables minute-level histories with over 160 frames, providing a $9\times \sim 40\times$ longer history horizon than prior methods. (c) **NATIVEMEM** achieves the highest success rates across memory-dependent manipulation tasks.

[†]Corresponding author: Hongyang Li hongyang@hku.hk

1 Introduction

Vision-Language-Action (VLA) models [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] extend Vision-Language Models [11, 12, 13, 14] to embodied decision-making, offering a promising path toward generalist robot control. However, most pretrained VLAs remain reactive, conditioning only on the current observation and instruction [15, 16, 17]. This single-frame setup is insufficient for memory-dependent manipulation, where actions may depend on task progress, prior interactions, counts, occluded states, or failures, requiring action-relevant visual history.

To address this challenge, recent works have explored two main paradigms for memory-augmented VLAs. The first builds memory outside the policy, where a high-level VLM retrieves past keyframes and plans sparse subtasks for a low-level VLA controller [18, 19, 20, 21, 22, 23, 24]. While effective for extending temporal context, such systems often turn memory into text format. However, such subtask descriptions require costly annotations for training, and subtle details are difficult to faithfully encode in language only. Another line of work builds memory with freshly initialized modules inside the policy, through recurrent states [25], compressed histories [26, 9, 27, 28, 29], or retrieval-augmented memory banks [30, 31, 32, 33, 34, 35]. Since these newly introduced modules are unseen during policy pretraining, this paradigm not only increases the overall architectural complexity, but also introduces the risk of performance degradation of the pretrained VLA. More fundamentally, they face a compression dilemma: without sufficient compression, fine-grained temporal histories are too costly to retain in full; with excessive or poorly structured compression, memory may discard details critical for action. This dilemma raises a central question: **how can a VLA retain long-horizon, fine-grained histories with minimal memory cost, while remaining compatible with the pretrained policy it builds upon?**

We address this question with **Native Memory Compression**, which compresses history directly into the pretrained VLA’s own visual token space. Instead of introducing a separate memory architecture, NATIVEMEM repurposes the VLA’s own vision encoder to distill past observations into compact memory tokens. We push compression to its token-efficient extreme: each historical frame is represented by a single native memory token, as shown in Fig. 1(a). Because these tokens are produced by the same visual pathway and injected into the original token sequence used by the pretrained policy, they can be interpreted as native visual evidence rather than external memory states, allowing the VLA to access long-horizon histories through its original attention mechanisms.

To learn such memory tokens so that they are both compact and aligned with the pretrained knowledge, we introduce a two-stage training pipeline that separates learning how to summarize memory from learning how to use it for a target task. In the first stage, we freeze the pretrained VLA and train a memory tokenizer derived from its native vision encoder. Given past observations, the tokenizer compresses each frame-view pair into a single visual summary token, which is appended to the original VLA token sequence. The tokenizer is optimized through the VLA’s native action prediction loss, making the summaries action-supervised rather than reconstruction-supervised. By training on a mixture of standard manipulation data and memory-demanding tasks, this stage learns a general memory tokenizer whose outputs remain aligned with the pretrained VLA’s token space. In the second stage, we freeze the learned tokenizer and perform task-specific full VLA finetuning with compact memory tokens, converting a pretrained single-frame VLA into a memory-augmented policy using limited task-specific demonstrations.

We extensively evaluate NATIVEMEM on manipulation tasks demanding long-horizon and adaptive memory understanding in both the simulation and the real world. Compared with pretrained memory-free VLA policies and prior memory-augmented baselines, NATIVEMEM achieves the highest task success rates while efficiently attending to minute-level histories with compact native memory, as shown in Fig. 1(b) and (c). Notably, even on unseen real-robot memory tasks, NATIVEMEM converts a pretrained single-frame VLA into a memory-augmented policy using only 100 task-specific demonstrations and approximately 5 hours of finetuning. Beyond this conversion efficiency, NATIVEMEM also exhibits strong data efficiency, achieving competitive performance with prior state-of-the-art methods while using only 20% of their training data.

2 Related Work

Vision-Language-Action Models. Vision-Language-Action (VLA) models [3, 4, 6, 36, 37, 38, 39, 40, 41, 42, 43, 44] extend Vision-Language Models (VLMs) [11, 12, 13, 45] to embodied decision making by grounding language instructions in visual observations and predicting robot actions. RT-1 [46], RT-2 [1], OpenVLA [2], and π_0 [3] have advanced the field through large-scale robot data, vision-language pretraining, open-source generalist policies, and continuous diffusion-based action generation. However, most VLA models condition on the current observation and short-term history. Autoregressive VLAs [2, 47, 48, 49] can encode previous visual, language, or action tokens as implicit memory, but such memory remains constrained by context length and lacks an explicit mechanism for preserving task-relevant information over long-horizon interactions.

VLM-Driven Memory with VLAs. To enhance memory in VLA models, recent methods often combine a high-level VLM-based memory or planning module with a low-level VLA controller [19, 20, 21, 50]. Representative examples include MemER [19], which retrieves task-relevant keyframes to guide a VLA policy via textual instructions; MEM [20], which combines short-term video memory with long-term language memory for progress tracking; and Mem-0 [21], which uses a VLM planner to generate subtasks for a diffusion-based executor. While effective, these methods rely on additional language-reasoning modules, increasing parameters and computation.

External Memory Modules in VLAs. To avoid the overhead of high-level memory or planning modules, recent works introduce external memory modules into VLA architectures [51, 52, 53, 54], mainly through explicit memory-bank retrieval or compressed history modeling. In the first direction, MemoryVLA [51] stores visual details and semantic tokens in a perceptual-cognitive memory bank, while MAP-VLA [52] builds a demonstration memory library with task-stage soft prompts. However, such methods often store selective information, rely on reliable memory construction and subtask segmentation, and may suffer from ambiguous boundaries or accumulated retrieval errors. Compressed history methods summarize past observations into compact representations. HAM-LET [26] encodes timesteps as moment tokens, ReMem-VLA [25] uses recurrent memory queries across frames and chunks, and MEM [20] compresses frame histories with a video encoder. Although these methods reduce the cost of full-history attention, they require architectural changes or dedicated design, limiting compatibility with pretrained VLAs and adding overhead as horizons grow. In contrast, NATIVEMEM encodes history using the pretrained VLA’s vision encoder, and reuses the policy’s existing attention with minimal overhead.

3 Method

3.1 Problem Formulation

We consider robotic manipulation in a partially observable environment. At control step t , a pretrained single-frame VLA policy maps the current multi-view observation $\mathbf{o}_t = \{o_t^v\}_{v=1}^V$ (with V camera views), proprioceptive state s_t , and language instruction ℓ to actions,

$$\mathbf{a}_t \sim \pi_\theta(\cdot \mid \mathbf{o}_t, s_t, \ell), \quad (1)$$

where θ denotes the parameters of the pretrained VLA. However, such reactive policies are fundamentally limited for long-horizon manipulation, which is often non-Markovian: the correct action depends not only on the current input but on the interaction history $\mathcal{H}_t$ available before step t . Crucially, identical current observations can demand different actions depending on what happened before. Formally, there may exist two histories $\mathcal{H}_t \neq \mathcal{H}'_t$ such that

$$\begin{aligned} \mathbf{o}_t &= \mathbf{o}'_t, \quad s_t = s'_t, \quad \ell = \ell', \\ \text{but } \mathbf{a}_t^*(\mathcal{H}_t, \mathbf{o}_t, s_t, \ell) &\neq \mathbf{a}_t^*(\mathcal{H}'_t, \mathbf{o}_t, s_t, \ell). \end{aligned} \quad (2)$$

This is the case whenever the robot must recall previously manipulated objects, operation counts, intermediate progress, or failed attempts. In all of these, the missing ingredient is historical information, and a policy that ignores it is under-specified by construction. We therefore convert a pretrained

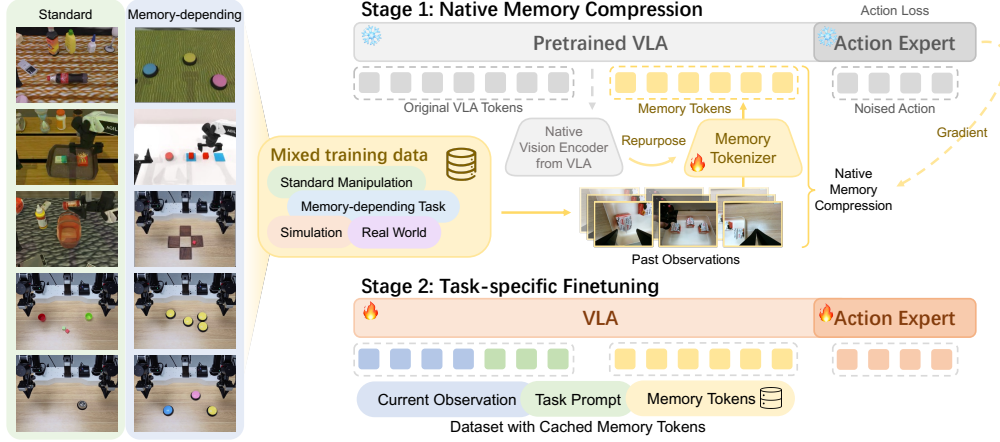


Figure 2: **Training pipeline of NATIVEMEM.** **Stage 1:** We freeze the pretrained VLA and learn a native memory tokenizer, initialized from its visual encoder, using the VLA’s original action prediction loss on mixed standard and memory-dependent manipulation data. The tokenizer compresses each frame-view observation into a single action-aligned memory token. **Stage 2:** We cache memory tokens and finetune the VLA with the original action loss by appending compressed memory tokens to the standard current-observation and prompt tokens.

single-frame VLA into a memory-augmented policy that additionally conditions on this history,

$$\mathbf{a}_t \sim \pi_{\theta'}(\cdot \mid \mathbf{o}_t, s_t, \ell, \mathcal{M}_t), \quad (3)$$

where $\mathcal{M}_t$ is a compact memory built from the history $\mathcal{H}_t$ and θ' are the adapted parameters. The central challenge is to design $\mathcal{M}_t$ to preserve action-relevant history while staying efficient and compatible with the pretrained VLA, motivating our native memory compression scheme.

3.2 Native Memory Compression via Action Supervision

Consider a policy that maintains a visual history over T seconds and updates memory at N frames per second. If each historical frame from each camera view is represented by M tokens, the memory length grows as

$$|\mathcal{M}_t| = V \cdot T \cdot N \cdot M, \quad (4)$$

where V is the number of camera views. This scaling reveals a fundamental **memory fidelity and efficiency trade-off**: increasing the temporal horizon T or the update frequency N improves the fidelity of historical context, but rapidly expands the VLA input sequence and its associated computation. Conversely, reducing the memory length improves efficiency, but risks discarding action-critical details. To retain fine-grained histories without overwhelming the VLA context, the per-frame token cost M must therefore be aggressively compressed.

Our key idea, **Native Memory Compression**, is to push this compression to an extreme by setting $M = 1$: each frame-view pair is summarized into a single memory token. However, compactness alone is insufficient. Since the memory tokens are consumed by a pretrained VLA, they must also be compatible with its native token space and action-generation prior. Arbitrary latent states from external modules may not be interpretable by the VLA, while summaries optimized for reconstruction or generic visual representation learning are not necessarily useful for action prediction. We therefore learn compact memory tokens directly through VLA’s original action objective.

To instantiate this idea, we derive a memory tokenizer from the VLA’s own visual encoder, as shown in Fig. 2. Given a frame-view observation $\mathbf{o}_t^v$, we initialize the memory encoder E_{mem} from the pretrained VLA visual encoder and introduce a learnable memory query token q_{mem} . The query token attends to the visual patch-token sequence $\mathbf{P}_\tau^v$ and aggregates it into a single summary token:

$$[\hat{q}_\tau^v, \hat{\mathbf{P}}_\tau^v] = E_{\text{mem}}([q_{\text{mem}}, \mathbf{P}_\tau^v]), \quad (5)$$

where $\hat{q}_\tau^v$ is the output memory summary. A linear memory projection maps this summary into the VLA token dimension by

$$m_\tau^v = W_{\text{mem}} \hat{q}_\tau^v. \quad (6)$$

The resulting m_τ^v can serve as a single native visual memory token for frame τ and view v .

Given a set of memory frame indices $\mathcal{I}_t$, we concatenate memory tokens across time and views, preceded by a learnable memory beginning-of-sequence token b_{mem} :

$$\mathcal{M}_t = [b_{\text{mem}}, \{m_\tau^v \mid \tau \in \mathcal{I}_t, \tau \leq t, v = 1, \dots, V\}]. \quad (7)$$

The memory sequence is appended to the original VLA input sequence,

$$\mathbf{x}_t = [\mathbf{x}_t^{\text{obs}}, \mathbf{x}^{\text{prompt}}(\ell, s_t), \mathcal{M}_t], \quad (8)$$

where $\mathbf{x}_t^{\text{obs}}$ denotes the current observation tokens and $\mathbf{x}^{\text{prompt}}(\ell, s_t)$ denotes the language and proprioceptive conditioning tokens. This introduces historical context without changing the VLA architecture: the policy attends to memory through its existing token-processing pipeline.

We learn these native memory tokens in a first-stage alignment procedure. The pretrained VLA is frozen, and only the memory branch is trainable. Given the augmented token sequence $\mathbf{x}_t$, the frozen VLA predicts the target action using its original action head, and the memory branch is optimized by the native VLA action loss:

$$\min_{\phi_{\text{mem}}, q_{\text{mem}}, W_{\text{mem}}, b_{\text{mem}}} \mathbb{E}_{(\mathcal{H}_t, \mathbf{o}_t, s_t, \ell, \mathbf{a}_t)} [\mathcal{L}_{\text{act}}(\pi_\theta(\cdot \mid \mathbf{o}_t, s_t, \ell, \mathcal{M}_t), \mathbf{a}_t)], \quad (9)$$

where θ is fixed and ϕ_{mem} denotes the parameters of the memory encoder. Since gradients can only update the memory branch, the learned tokens are encouraged to encode information that is both action-relevant and aligned with the frozen VLA’s pretrained token space.

For training efficiency, we do not load the full visual history. Instead, for each training step, $\mathcal{I}_t$ consists of the first frame of the episode and a recent history window. The first frame provides coarse task initialization context, while the recent window captures state changes and progress. We also include the current frame in $\mathcal{I}_t$, so that the tokenizer learns a unified frame-level summarization behavior for both current and past observations. Our memory tokenizer is trained on a mixture of simulation and real-world demonstrations covering both standard manipulation data and memory-demanding tasks, and could be reused for downstream memory-augmented finetuning.

3.3 Task-Specific Finetuning and Real-Time Memory Inference

Task-specific finetuning. Given a target-task demonstration dataset, we first use the learned memory tokenizer to preprocess the visual history offline. For each episode, every frame from each camera view is converted into its corresponding memory summary token $\hat{q}_\tau^v$. Since each frame-view pair is represented by only one token, this preprocessing introduces negligible storage and I/O overhead compared with storing dense visual token sequences.

For each training step t , we retrieve the cached summary tokens from selected frame indices $\mathcal{I}_t$ with $\tau \leq t$, and form the memory sequence following Eq. 7. The resulting memory sequence is appended to the standard VLA input as in Eq. 8.

We initialize the VLA backbone from the pretrained single-frame policy and load W_{mem} and b_{mem} from the first-stage memory branch. The memory tokenizer encoder is kept fixed during this stage, while the VLA backbone, action head, and memory projection are finetuned on limited target-task demonstrations using the native action prediction loss:

$$\min_{\theta', W_{\text{mem}}, b_{\text{mem}}} \mathbb{E}_{(\mathbf{o}_t, s_t, \ell, \mathcal{M}_t, \mathbf{a}_t)} [\mathcal{L}_{\text{act}}(\pi_{\theta'}(\cdot \mid \mathbf{o}_t, s_t, \ell, \mathcal{M}_t), \mathbf{a}_t)]. \quad (10)$$

Here, θ' denotes the finetuned VLA parameters. This preserves the standard VLA finetuning workflow while extending the policy with compact historical context.

Real-time memory inference. During deployment, the memory tokenizer can operate independently from the VLA policy. As new observations arrive, the tokenizer converts them into memory

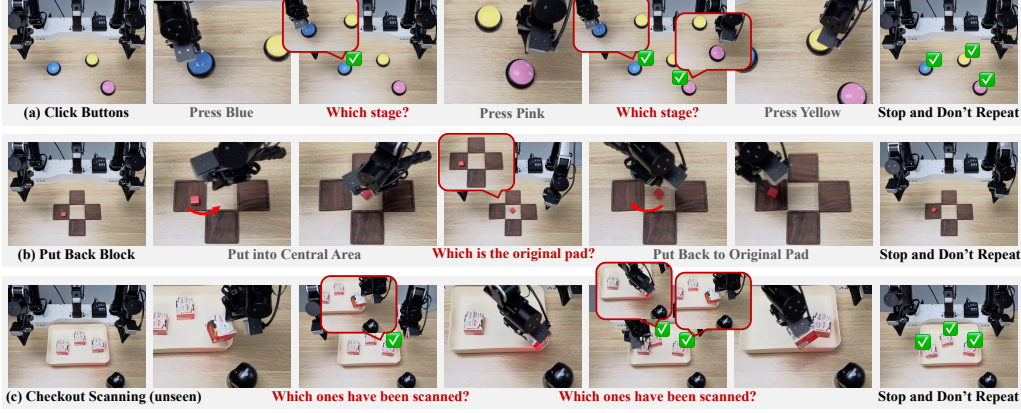


Figure 3: **Memory-dependent manipulation tasks.** In *Click Buttons*, the robot must follow a specified button sequence without repeating completed presses. In *Put Back Block*, it must remember the block’s original pad after moving it to the center. In *Grocery Checkout Scanning*, it must track which items have already been scanned and avoid duplicate or missed scans.

tokens at a specified update frequency and maintains a compact memory queue. Since the tokenizer is derived from the original visual encoder and produces only one token per frame-view pair, memory updates can be performed with low overhead and do not require modifying the VLA inference pipeline. When the VLA is queried for an action, the current observation, proprioceptive state, and language instruction are processed as usual. The current memory queue is simply concatenated after the original input tokens, following Eq 8. Thus, memory construction and policy inference are decoupled: the tokenizer can update historical memory at a high frequency, while the VLA consumes the latest compact memory sequence whenever action prediction is required.

4 Experiments

4.1 Experimental Setup

Datasets and Tasks. We evaluate NATIVEMEM on memory-dependent manipulation tasks in both simulation and the real world. For real-world evaluation, we consider three tasks, as illustrated in Fig. 3. Notably, *Grocery Checkout Scanning (unseen)* is not included during first-stage memory-tokenizer training. This setting evaluates whether the learned native memory representation transfers to new forms of long-horizon task progress tracking. In simulation, we use three RMBench tasks [21] and two additional button-pressing tasks. The simulated *Click Buttons* task follows the same memory requirement as its real-world counterpart, while *Click Buttons (Hard)* removes color distinctions between buttons, forcing the policy to rely on spatial memory and interaction history.

Baselines. We compare NATIVEMEM with representative VLA policies that instantiate different memory-modeling paradigms. For *VLM-driven memory*, we include MemER [19] and Mem-0 [21]. For *external memory modules*, we include HAMLET [26] and MEM-short [20]. For a fair comparison, we implement both HAMLET and MEM-short on the same $\pi_{0.5}$ backbone used by NATIVEMEM. Since HAMLET and MEM-short are not publicly released, we faithfully reproduce them following the official papers. MEM-short is originally designed to be retrained on large-scale data. To ensure a controlled comparison, we train it using the same demonstration trajectories available to all methods. Therefore mark them as HAMLET* and MEM-short* in Tab. 1 and 2.

4.2 Main Results

Simulation Experiments. As shown in Tab. 1, NATIVEMEM achieves the best performance across all tasks, improving the average success rate to 84.0%. The evaluated tasks cover two complementary memory requirements: long-range recall of early observations, as in *Put Back Block* and

Method	Click Buttons	Click Buttons (hard)	Swap Blocks	Put Back Block	Observe and Pickup	Avg.
<i>Base VLA Policies</i>						
$\pi_{0.5}$ [4]	0	0	24	11	9	8.8
X-VLA [55]	7	12	16	18	9	12.4
<i>VLM-Driven Memory</i>						
MemER [19]	12	8	18	12	2	10.4
Mem-0 [21]	0	1	67	90	4	32.4
<i>External Memory Modules</i>						
HAMLET* [26]	4	17	11	3	10	9.0
MEM-short* [20]	0	39	4	15	6	12.8
Ours	94	88	94	100	44	84.0

Table 1: Task success rates (%) in simulation evaluation. The benchmark includes three RMBench tasks and two additional simulated button-pressing tasks. NATIVEMEM achieves the best performance across all tasks. The best result for each task is marked in **bold**. * indicates reproduced baselines implemented on the same $\pi_{0.5}$ backbone.

Method	Click Buttons			Put Back Block		Grocery Checkout Scanning (unseen)			Avg.
	S1	S2	S3	S1	S2	S1	S2	S3	
$\pi_{0.5}$ [4]	16	8	2	72	14	90	58	28	14.7
$\pi_{0.5}$ + RTC [56]	26	18	16	56	24	74	66	64	34.7
Mem-0 [21]	36	4	0	0	0	6	0	0	0.0
MEM-short* [20]	72	52	40	6	0	72	56	52	30.7
Ours	100	98	96	100	100	100	100	100	98.7

Table 2: Real-world cumulative task success rates (%). S1-S3 denote cumulative success after each task stage; the last stage corresponds to the overall task success rate. *Grocery Checkout Scanning* is unseen during first-stage memory-tokenizer training. The mean overall success rate is reported. The best results are marked in **bold**. * denotes reproduced baselines trained with the same number of task-specific demonstrations as NATIVEMEM.

Observe and Pickup, and continuous progress tracking, as in *Click Buttons*, *Click Buttons (Hard)*, and *Swap Blocks*. Existing VLM-driven or short-horizon memory methods struggle with one or both settings, while NATIVEMEM remains consistently effective, suggesting its native memory compression provides a unified representation for both recalling visual evidence and tracking task states.

Real-World Experiments. Tab. 2 shows that NATIVEMEM also transfers effectively to real-robot manipulation, achieving the highest overall success rate across all tasks. While RTC improves over the $\pi_{0.5}$ policy by exploiting temporal action continuity, rather than explicitly remembering past observations. Its performance still drops in later stages that require explicit historical recall. MEM-short* becomes less stable under limited real-world data, occasionally degrading the pre-trained VLA’s manipulation capability. In contrast, NATIVEMEM preserves the pretrained policy prior while adding compact history, achieving the strongest performance across all three tasks.

Inference Efficiency. Fig. 4(a) compares inference latency and peak GPU memory under increasing history length. Our NATIVEMEM supports up to 5,000 historical frames within a 32 GB memory budget, and still attends to about 200 frames under a 100 ms real-time latency constraint, enabling substantially longer histories while preserving the real-time reactivity.

Data-Efficiency. Fig. 4(b) shows that NATIVEMEM adapts effectively with limited demonstrations. With only 10 demonstrations, it reaches 60% average success, 3.5× higher than Mem-0, and consistently outperforms Mem-0 with 25 and 50 demonstrations, reflecting the benefit of reusing pretrained VLA manipulation priors.

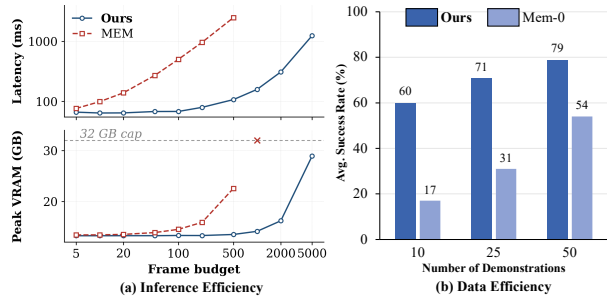


Figure 4: (a) Inference latency and memory consumption. (b) Average data efficiency on three RMBench tasks.

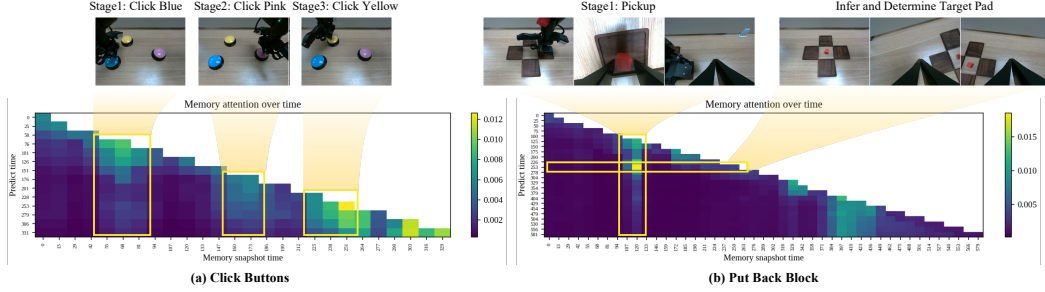


Figure 6: Visualization of attention from action tokens to historical memory snapshots. The x-axis denotes the timestamp of each memory snapshot, and the y-axis denotes the action prediction timestep. Each row shows which memories are used for a given action prediction time, while each column indicates how a specific memory is attended by future predictions. The attention in *Click Buttons* follows the three clicking stages, while in *Put Back Block* it concentrates on the pickup moment needed to infer the target pad.

Method	Click Button	Click Button (hard)	Swap Blocks	Put Back Block	Observe and Pickup	Avg.
Unfrozen VLA	38	24	45	0	9	23.2
w/o Stage1	94	80	92	17	7	58.0
Sparse update (0.5Hz)	83	63	91	24	9	53.8
Short horizon (2s)	26	87	0	18	39	34.0
Short horizon (4s)	69	87	0	18	37	43.0
Short horizon (6s)	80	88	0	18	44	46.0
Ours	94	88	94	100	44	84.0

Table 3: **Ablation Study on Native Memory Compression and its Temporal Coverage.** The first two variants evaluate the memory-compression learning, while the remaining variants examine the importance of long-horizon and fine-grained history. The best results are marked in **bold**.

4.3 Qualitative Analysis: What Does Native Memory Capture?

Spatial Attention of the Memory Tokenizer.

To understand what is encoded into each memory token, we visualize the tokenizer’s spatial attention over historical observations. As shown in Fig. 5, the tokenizer consistently focuses on manipulation-relevant regions rather than background pixels. For *Put Back Blocks*, attention concentrates on the block and its corresponding pad. Notably, this behavior also generalizes to *Grocery Checkout Scanning*, which is unseen during tokenizer training: the tokenizer still attends to foreground objects, especially the item about to be grasped.

Action Attention over the Memory Sequence.

To examine how NATIVEMEM uses history during action generation, we visualize the attention from action tokens to memory tokens across inference time. Fig. 6 shows that the policy attends to task-relevant moments rather than simply the most recent observations. In *Click Buttons*, high-attention memories align with the three timesteps when individual buttons were pressed. In *Put Back Block*, attention concentrates on the moment when the block was lifted from its original pad, which is critical for deciding where to return it.

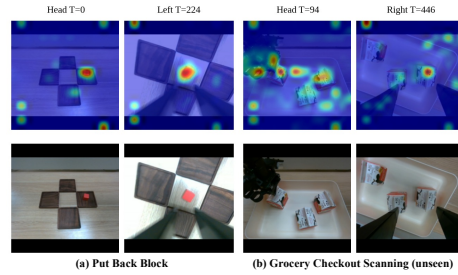


Figure 5: Visualization of the spatial attention used by the memory tokenizer when compressing each frame-view observation into a single memory token. The top row shows the tokenizer attention maps over input images, while the bottom row shows the corresponding raw observations.

4.4 Ablation Study

Native Memory Compression. As shown in Tab. 3 (lines 1 and 2), when the VLA is unfrozen during memory-compression learning, performance drops to 23.2%. The model reduces the action

loss by directly adapting the pretrained VLA itself rather than forcing the memory branch. After skipping Native Memory Compression, mean-pooled vision-encoder features are used as memory tokens. The results show that such generic visual features can capture coarse historical context, but fail to preserve details required by tasks such as *Put Back Block* and *Observe and Pickup*. These results indicate that Native Memory Compression is essential for distilling task-relevant historical cues, enabling the policy to recover critical information needed for downstream action prediction.

Temporal Memory Coverage. Sparse update at 0.5Hz broadly reduces success rates, showing that the policy needs fine-grained temporal evidence to track interaction states. Short horizons degrade tasks whose critical information falls outside the retained window, such as *Swap Blocks* and *Put Back Block*. Results in Tab. 3 (lines 4~6) show that NATIVEMEM’s performance relies on both dense temporal coverage and long-horizon retention: it updates memory frequently enough to capture fine-grained interaction changes, while its compact native tokens make minute-level history retention scalable.

5 Conclusion

We presented NATIVEMEM, which enables pretrained single-frame VLAs to retain long-horizon, fine-grained visual histories through **Native Memory Compression**. By repurposing the VLA’s own vision encoder, NATIVEMEM achieves one-token-per-frame compression. We further introduced a two-stage training pipeline that first learns an action-supervised memory tokenizer aligned with the pretrained VLA’s visual-action priors, and then performs task-specific finetuning with limited demonstrations. Across simulation and real-world manipulation tasks, NATIVEMEM substantially improves average success rates from 32.4% to 84.0% in simulation and from 34.7% to 98.7% on real robots, while matching leading memory-designed methods with only 20% of the training data.

6 Limitations

While NATIVEMEM enables dense minute-level visual memory for long-horizon manipulation, it is not designed to maintain persistent memories over hours or days. Supporting such long-term continuity will likely require additional system-level memory infrastructure. In addition, our memory tokenizer is learned solely through action supervision. Although we have not observed clear failures in our preliminary exploration, more complex tasks may expose a semantic gap between low-level action losses and higher-level, abstract memory requirements. Exploring more direct and scalable objectives for learning the relationship between memory and action remains an important direction.

References

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, 2023.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [4] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with Open-World generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [5] Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li. UniVLA: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025.
- [6] G. Team, A. Ye, B. Wang, C. Ni, G. Huang, G. Zhao, H. Li, J. Li, J. Zhu, L. Feng, et al. GigaBrain-0: A world model-powered vision-language-action model. *arXiv preprint arXiv:2510.19430*, 2025.
- [7] C. Ni, C. Chen, X. Wang, Z. Zhu, W. Zheng, B. Wang, T. Chen, G. Zhao, H. Li, Z. Dong, et al. SwiftVLA: Unlocking spatiotemporal dynamics for lightweight vla models at minimal overhead. *CVPR 2026*, 2026.
- [8] A. Ye, B. Wang, C. Ni, G. Huang, G. Zhao, H. Li, H. Li, J. Li, J. Lv, J. Liu, et al. GigaWorld-Policy: An efficient action-centered world-action model. *arXiv preprint arXiv:2603.17240*, 2026.
- [9] H. Jang, S. Yu, H. Kwon, H. Jeon, Y. Seo, and J. Shin. ContextVLA: Vision-Language-Action model with amortized multi-frame context. *arXiv preprint arXiv:2510.04246*, 2025.
- [10] J. Zhang, Y. Chen, Y. Xu, Z. Huang, Y. Zhou, Y.-J. Yuan, X. Cai, G. Huang, X. Quan, H. Xu, et al. 4D-VLA: Spatiotemporal vision-language-action pretraining with cross-scene calibration. *arXiv preprint arXiv:2506.22242*, 2025.
- [11] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Alabdulmohsin, M. Tschanen, E. Bugliarello, et al. PaliGemma: A versatile 3b VLM for transfer. *arXiv preprint arXiv:2407.07726*, 2024.
- [12] A. Marafioti, O. Zohar, M. Farré, M. Noyan, E. Bakouch, P. Cuenca, C. Zakka, L. B. Allal, A. Lozhkov, N. Tazi, et al. SmolVLM: Redefining small and efficient multimodal models. *arXiv preprint arXiv:2504.05299*, 2025.
- [13] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [14] S. Zhou, A. Vilesov, X. He, Z. Wan, S. Zhang, A. Nagachandra, D. Chang, D. Chen, X. E. Wang, and A. Kadambi. VLM4D: Towards spatiotemporal awareness in vision language models. In *Proceedings of the IEEE/CVF international conference on computer vision*, 2025.
- [15] Z. He, Y. Wang, C. Zhi, Y. Hu, T.-P. Chen, L. Yin, Z. Chen, T. A. Wu, S. Ouyang, Z. Wang, et al. MemoryArena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint arXiv:2602.16313*, 2026.

- [16] M. Lauri, D. Hsu, and J. Pajarinen. Partially observable markov decision processes in robotics: A survey. *IEEE Transactions on Robotics*, 2022.
- [17] Y. Wu, S. Liang, C. Zhang, Y. Wang, Y. Zhang, H. Guo, R. Tang, and Y. Liu. From human memory to AI memory: A survey on memory mechanisms in the era of LLMs. *arXiv preprint arXiv:2504.15965*, 2025.
- [18] M. Zhai, Z. Gao, Y. Wu, and Y. Jia. Memory-Centric embodied question answering. *arXiv preprint arXiv:2505.13948*, 2025.
- [19] A. Sridhar, J. Pan, S. Sharma, and C. Finn. MemER: Scaling up memory for robot control via experience retrieval. *arXiv preprint arXiv:2510.20328*, 2025.
- [20] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, K. Stachowicz, et al. MEM: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.
- [21] T. Chen, Y. Wang, M. Li, Y. Qin, H. Shi, Z. Li, Y. Hu, Y. Zhang, K. Wang, Y. Chen, et al. RM-Bench: Memory-Dependent robotic manipulation benchmark with insights into policy design. *arXiv preprint arXiv:2603.01229*, 2026.
- [22] Y. Wang, Z. Gu, Y. Gao, A. Jiang, Z. Sun, S. Wang, Y. Heng, and H. Sun. HiST-VLA: A hierarchical spatio-temporal vision-language-action model for end-to-end autonomous driving. *arXiv preprint arXiv:2602.13329*, 2026.
- [23] H. Wang, Z. Jing, J. Ao, S. Song, X. Li, G. Huang, and C. Bai. Beyond Short-Horizon: VQ-Memory for robust long-horizon manipulation in non-markovian simulation benchmarks. *arXiv preprint arXiv:2603.09513*, 2026.
- [24] Z. Zeng, F. Ding, H. Yang, and X. Li. HELM: Harness-Enhanced long-horizon memory for vision-language-action manipulation. *arXiv preprint arXiv:2604.18791*, 2026.
- [25] H. Li, F. Shen, D. Chen, L. Yang, X. Wang, J. Shi, Z. Bing, Z. Liu, and A. Knoll. ReMem-VLA: Empowering vision-language-action model with memory via dual-level recurrent queries. *arXiv preprint arXiv:2603.12942*, 2026.
- [26] M. Koo, D. Choi, T. Kim, K. Lee, C. Kim, Y. Seo, and J. Shin. HAMLET: Switch your vision-language-action model into a history-aware policy. *arXiv preprint arXiv:2510.00695*, 2025.
- [27] M. Lin, P. Ding, S. Wang, Z. Zhuang, Y. Liu, X. Tong, W. Song, S. Lyu, S. Huang, and D. Wang. HiF-VLA: Hindsight, Insight and Foresight through motion representation for Vision-Language-Action models. *arXiv preprint arXiv:2512.09928*, 2025.
- [28] J. Liu, Y. Qi, J. Zhang, M. Li, S. Wang, K. Wu, H. Ye, H. Zhang, Z. Chen, F. Zhong, et al. TrackVLA++: Unleashing reasoning and memory capabilities in VLA models for embodied visual tracking. *arXiv preprint arXiv:2510.07134*, 2025.
- [29] G. Nangue Tasse, M. Riemer, B. Rosman, and T. Klinger. Beyond sliding windows: Learning to manage memory in non-markovian environments. *arXiv e-prints*, 2025.
- [30] H. Fang, M. Grotz, W. Pumacay, Y. R. Wang, D. Fox, R. Krishna, and J. Duan. SAM2Act: Integrating visual foundation model with a memory architecture for robotic manipulation. *arXiv preprint arXiv:2501.18564*, 2025.
- [31] Y. Yang, H. Yang, J. Zhou, P. Chen, H. Zhang, Y. Du, and C. Gan. 3D-mem: 3D scene memory for embodied exploration and reasoning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025.

- [32] Y. Gao, J. Liu, S. Li, and S. Song. Gated memory policy. *arXiv preprint arXiv:2604.18933*, 2026.
- [33] Z. Chen, Y. Hu, Z. Fu, Z. Li, J. Huang, Q. Huang, and Y. Wei. INTENT: Invariance and discrimination-aware noise mitigation for robust composed image retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [34] Z. Li, Y. Hu, Z. Chen, S. Zhang, Q. Huang, Z. Fu, and Y. Wei. HABIT: Chrono-synergia robust progressive learning framework for composed image retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [35] Z. Li, Y. Hu, Z. Chen, Q. Huang, G. Qiu, Z. Fu, and M. Liu. ReTrack: Evidence-driven dual-stream directional anchor calibration network for composed video retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [36] A. Zhai, B. Liu, B. Fang, C. Cai, E. Ma, E. Yin, H. Wang, H. Zhou, J. Wang, L. Shi, et al. Igniting VLMs toward the embodied space. *arXiv preprint arXiv:2509.11766*, 2025.
- [37] J. Yang, K. Lin, J. Li, W. Zhang, T. Lin, L. Wu, Z. Su, H. Zhao, Y.-Q. Zhang, L. Chen, et al. RISE: Self-improving robot policy with compositional world model. *arXiv preprint arXiv:2602.11075*, 2026.
- [38] G. Team, A. Ye, B. Wang, C. Ni, G. Huang, G. Zhao, H. Li, J. Zhu, K. Li, M. Xu, et al. GigaWorld-0: World models as data engine to empower embodied ai. *arXiv preprint arXiv:2511.19861*, 2025.
- [39] G. Team, B. Wang, B. Li, C. Ni, G. Huang, G. Zhao, H. Li, J. Li, J. Lv, J. Liu, et al. GigaBrain-0.5M*: a vla that learns from world model-based reinforcement learning. *arXiv preprint arXiv:2602.12099*, 2026.
- [40] A. Ye, Z. Zhang, B. Wang, X. Wang, D. Zhang, and Z. Zhu. VLA-R1: Enhancing reasoning in vision-language-action models. *arXiv preprint arXiv:2510.01623*, 2025.
- [41] H. Li, I. Zhang, R. Ouyang, X. Wang, Z. Zhu, Z. Yang, Z. Zhang, B. Wang, C. Ni, W. Qin, et al. MimicDreamer: Aligning human and robot demonstrations for scalable VLA training. *arXiv preprint arXiv:2509.22199*, 2025.
- [42] Y. Li, L. Zhou, S. Yan, B. Liao, T. Yan, K. Xiong, L. Chen, H. Xie, B. Wang, G. Chen, et al. UniDriveVLA: Unifying understanding, perception, and action planning for autonomous driving. *arXiv preprint arXiv:2604.02190*, 2026.
- [43] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, et al. TinyVLA: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.
- [44] W. Zhang, H. Liu, Z. Qi, Y. Wang, X. Yu, J. Zhang, R. Dong, J. He, H. Wang, Z. Zhang, et al. DreamVLA: a vision-language-action model dreamed with comprehensive world knowledge. *arXiv preprint arXiv:2507.04447*, 2025.
- [45] B. Chen, Z. Xu, S. Kirmani, B. Ichter, D. Sadigh, L. Guibas, and F. Xia. SpatialVLM: Endowing vision-language models with spatial reasoning capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [46] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [47] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

- [48] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang, et al. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.
- [49] Y. Hu, J.-N. Zaeche, N. Nikolov, Y. Yao, S. Dey, G. Albanese, R. Detry, L. Van Gool, and D. Paudel. AR-VLA: True autoregressive action expert for Vision-Language-Action models. *arXiv preprint arXiv:2603.10126*, 2026.
- [50] F. Peller-Konrad, R. Kartmann, C. R. Dreher, A. Meixner, F. Reister, M. Grotz, and T. Asfour. A memory system of a robot cognitive architecture and its implementation in ArmarX. *Robotics and Autonomous Systems*, 2023.
- [51] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang. MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2508.19236*, 2025.
- [52] R. Li, W. Guo, Z. Wu, C. Wang, H. Deng, Z. Weng, Y.-P. Tan, and Z. Wang. MAP-VLA: Memory-augmented prompting for vision-language-action model in robotic manipulation. *arXiv preprint arXiv:2511.09516*, 2025.
- [53] M. Neau, Z. Falomir, P. E. Santos, A.-G. Bossier, and C. Buche. GraSP-VLA: Graph-based symbolic action representation for long-horizon planning with VLA policies. *arXiv preprint arXiv:2511.04357*, 2025.
- [54] X. Guo, C. Jiang, H. B. Kim, Y. Sun, Y. Xiao, Y. Han, and J. Yang. Chameleon: Episodic memory for long-horizon robotic manipulation. *arXiv preprint arXiv:2603.24576*, 2026.
- [55] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, et al. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.
- [56] K. Black, A. Z. Ren, M. Equi, and S. Levine. Training-time action conditioning for efficient real-time chunking. *arXiv preprint arXiv:2512.05964*, 2025.